

SHOPEZ

E-Commerce Application

Project Documentation

MERN Stack Full-Stack Development Project
MongoDB · Express.js · React.js · Node.js

Table of Contents

1. Project Overview 3

2. MVC Architectural Pattern..... 3

3. Entity-Relationship (ER) Model..... 4

4. Database Schema (MongoDB Collections)..... 6

5. Key Features 6

6. Roles and Responsibilities..... 6

7. User Flow..... 7

8. Development Roadmap 7

9. Summary 8

1. Project Overview

SHOPEZ is a full-stack e-commerce web application built on the MERN stack (MongoDB, Express.js, React.js, and Node.js). The platform enables users to register, browse a product catalog, add items to a cart, and place orders, while administrators manage banners and product categories through a dedicated admin interface.

Layer	Technology
Frontend	React (UI Layer)
Backend	Node.js + Express.js
Database	MongoDB with Mongoose ODM

1.1 High-Level Architecture

The application follows a standard three-tier architecture:

Frontend (React) → Backend (Express / Node) → Database (MongoDB)

- Frontend — handles UI components such as User Authentication, Cart, Products, Profile, and Admin Dashboard.
- Backend — exposes REST API endpoints for Users, Products, Cart, and Orders, and manages Admin authentication.
- Database — stores collections for Users, Cart, Orders, and Products.

2. MVC Architectural Pattern

The Shop-EZ backend follows the Model-View-Controller (MVC) architectural pattern — a software design approach that separates the application into three interconnected layers, allowing for modularity, easier maintenance, and scalability.

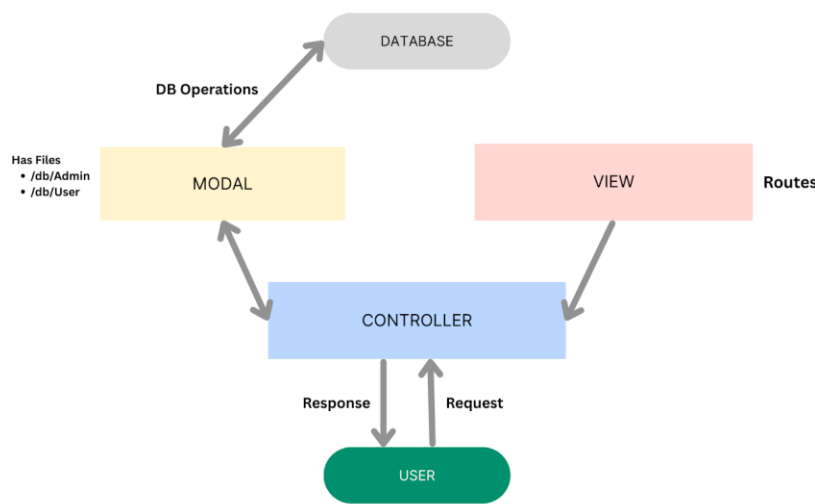


Figure 1: MVC Architecture of the SHOPEZ Backend

2.1 Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic, including the definition of data schemas and the operations performed on the database using those schemas. Models are implemented using Mongoose, which provides a schema-based solution for modeling application data in MongoDB. Model files are organized under /db/Admin and /db/User.

2.2 Controller Layer

The Controller layer acts as an intermediary between the View (routes) and the Model. It receives incoming requests, processes the input (including validation or transformation), calls the appropriate methods from the model, and returns a response to the client.

2.3 View Layer (Routing Layer)

In the context of a backend REST API, the View is implemented as the routing layer, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

2.4 Advantages of Using MVC in This Project

- Separation of Concerns — each layer has a clearly defined responsibility, improving readability and maintainability.
- Scalability — new features can be added easily by creating new routes, controllers, and models.
- Reusability — logic in controllers and models can be reused across multiple parts of the application.
- Testing — each layer can be tested independently, especially controllers and models.
- Collaboration-Friendly — multiple developers can work simultaneously on different layers without conflict.

3. Entity-Relationship (ER) Model

The ER diagram below defines the core entities of the SHOPEZ platform and their attributes.

ER-MODEL

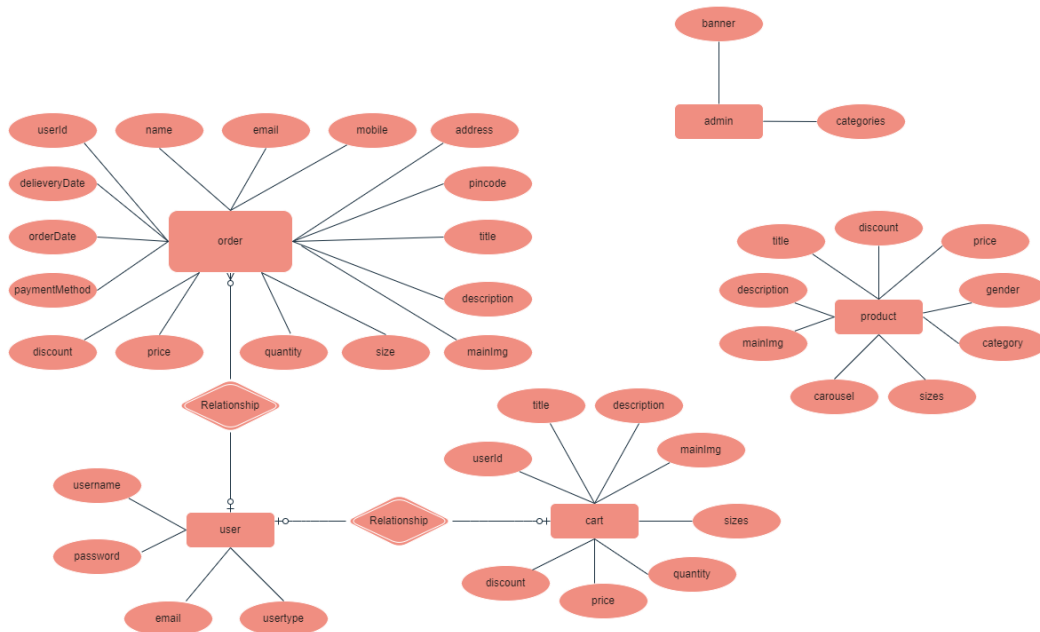


Figure 2: SHOPEZ ER Model

3.1 Entity Descriptions

Entity	Description
User	Represents the individuals or entities registered on the platform.
Admin	Manages platform-level content such as banner images and categories.
Product	Represents the collection of all products available for sale.
Cart	Stores products added by users; entries are differentiated by userId.
Order	Stores all orders placed by users on the platform.

3.2 Entity Attributes

Entity	Attributes
User	userId, username, password, email, usertype
Admin	banner, categories
Product	title, description, mainImg, carousel, category, sizes, gender, price, discount
Cart	userId, title, description, mainImg, sizes, quantity, price, discount
Order	userId, name, email, mobile, address, pincode, title, description, mainImg, size, quantity, price, discount, paymentMethod, orderDate, deliveryDate

4. Database Schema (MongoDB Collections)

The database schema translates the ER model into concrete MongoDB collections, shown below.

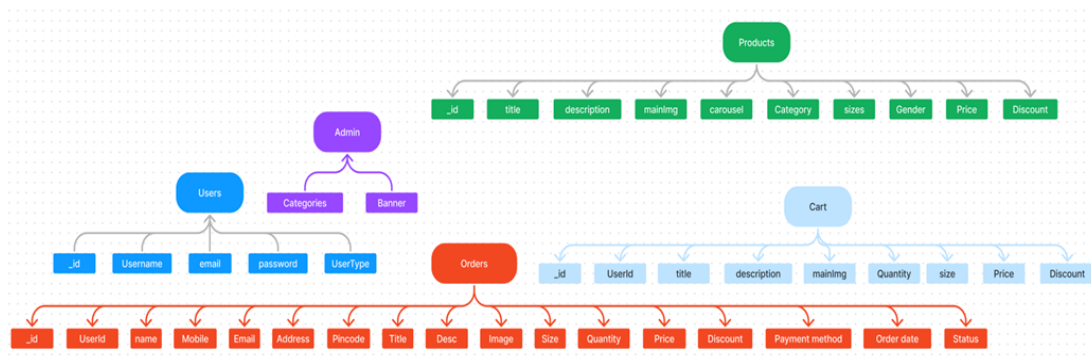


Figure 3: SHOPEZ Database Schema

Collection	Fields
Users	_id, Username, email, password, UserType
Admin	Categories, Banner
Products	_id, title, description, mainImg, carousel, Category, sizes, Gender, Price, Discount
Cart	_id, UserId, title, description, mainImg, Quantity, size, Price, Discount
Orders	_id, UserId, name, Mobile, Email, Address, Pincode, Title, Desc, Image, Size, Quantity, Price, Discount, Payment method, Order date, Status

5. Key Features

1. Comprehensive Product Catalog — ShopEZ offers an extensive catalog of products, letting shoppers explore items with detailed descriptions, customer reviews, pricing, and discounts.
2. Shop Now Button — Each product listing features a convenient "Shop Now" button to initiate the purchasing process.
3. Order Details Page — Clicking "Shop Now" directs the user to an order details page to provide shipping address, preferred payment method, and any specific product requirements.
4. Secure & Efficient Checkout — Personal information is handled securely, with a purchasing process designed to be swift and trouble-free.
5. Order Confirmation & Details — After placing an order, users receive a confirmation and can review full order details, including shipping information, payment method, and product requests, on an order details page.

6. Roles and Responsibilities

Role	Responsibilities
User	Register and log in, browse the product catalog, manage cart items, place orders, and view order history in the profile section.

Role	Responsibilities
Admin	Manage banner images and product categories, and oversee catalog content through the admin dashboard.

7. User Flow

1. Users start by registering for an account.
2. After registration, they can log in with their credentials.
3. Once logged in, they can browse the available products on the platform.
4. Users add the products they wish to purchase to their cart and proceed to order.
5. They then enter address and payment details to complete checkout.
6. After ordering, users can review their orders in the profile section.

Register → Login → Browse Products → Add to Cart → Enter Address / Payment → Place Order → View Orders in Profile

8. Development Roadmap

The recommended build sequence for the SHOPEZ project is as follows:

Step 1 — Project Architecture (Planning Phase)

Finalize the Technical Architecture, ER Diagram, Feature list, Roles, User Flow, and MVC pattern, as documented in the sections above.

Step 2 — Project Setup and Configuration

- Initialize the backend project (npm init) and install Express, Mongoose, and other dependencies.
- Initialize the frontend React application.
- Set up the MongoDB connection (local instance or MongoDB Atlas).
- Configure environment variables (.env) for database URI, port numbers, and secrets.

Step 3 — Database Development

- Create Mongoose schemas for User, Product, Cart, Order, and Admin, based on the attributes defined in Sections 3 and 4.

Step 4 — Backend Development

- Build Models for each collection.
- Build Controllers containing business logic (e.g., register/login, product CRUD, add-to-cart, place-order, banner/category management).
- Build Routes (Views) to expose REST API endpoints for each controller.
- Implement authentication for both Users and Admin.

Step 5 — Frontend Development

- Build React components/pages matching the User Flow: registration & login, product listing & details, cart, checkout (address + payment), order confirmation, and order history (profile).
- Build the Admin Dashboard for managing banners, categories, and products.
- Connect frontend components to backend REST API endpoints.

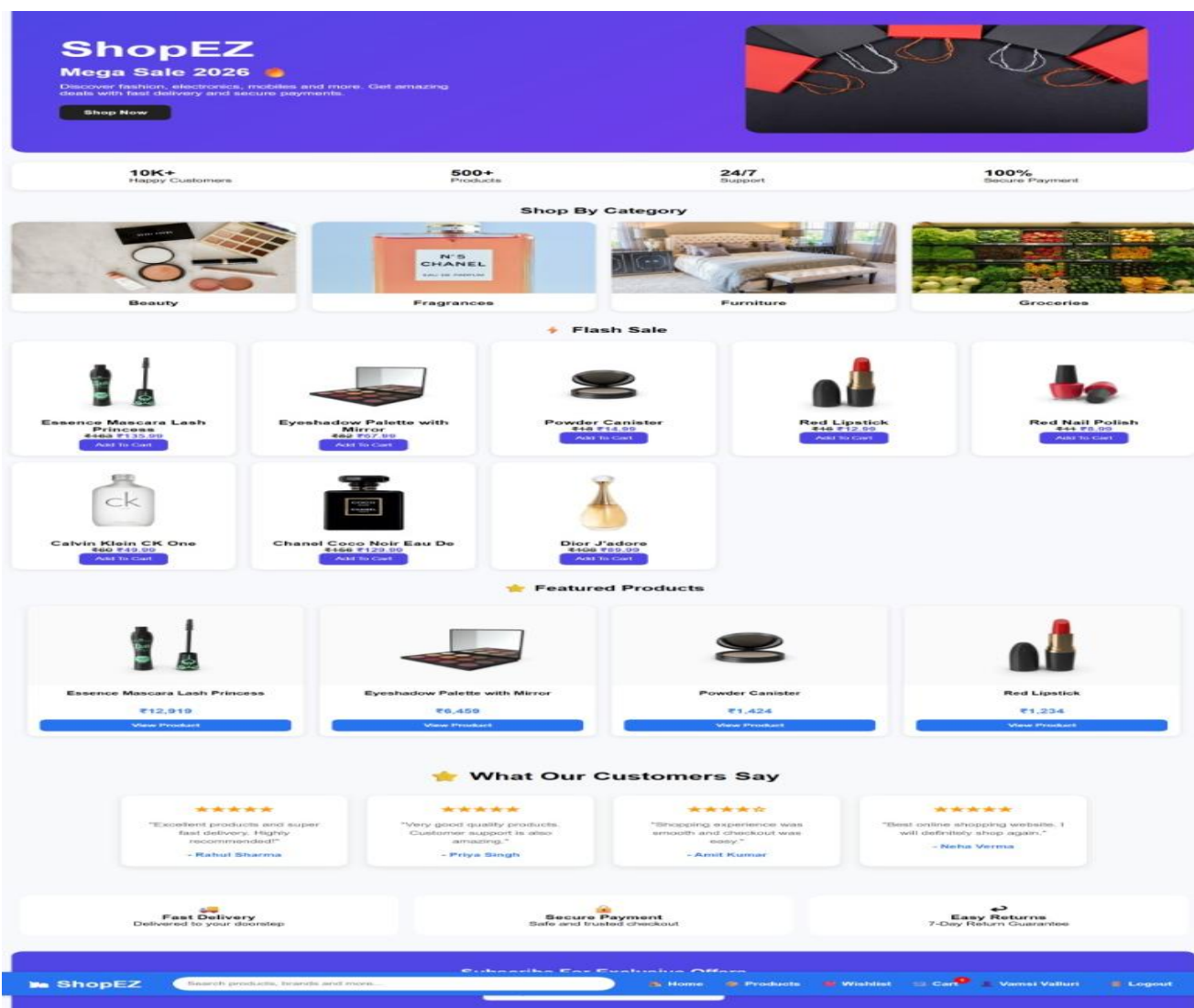
Step 6 — Project Execution

- Test the complete flow end-to-end (registration through order confirmation).
- Fix bugs, refine the UI, and deploy the application.

PROJECT OVERVIEW

Home Page

-The Home Page serves as the entry point of the SHOPEZ application. It displays featured products, categories, promotional banners, and navigation options, allowing users to explore the platform efficiently.



User Registration Page

-This page enables new users to create an account by providing personal details such as username, email, and password. Input validation ensures secure and accurate registration.

Create Account

Full Name

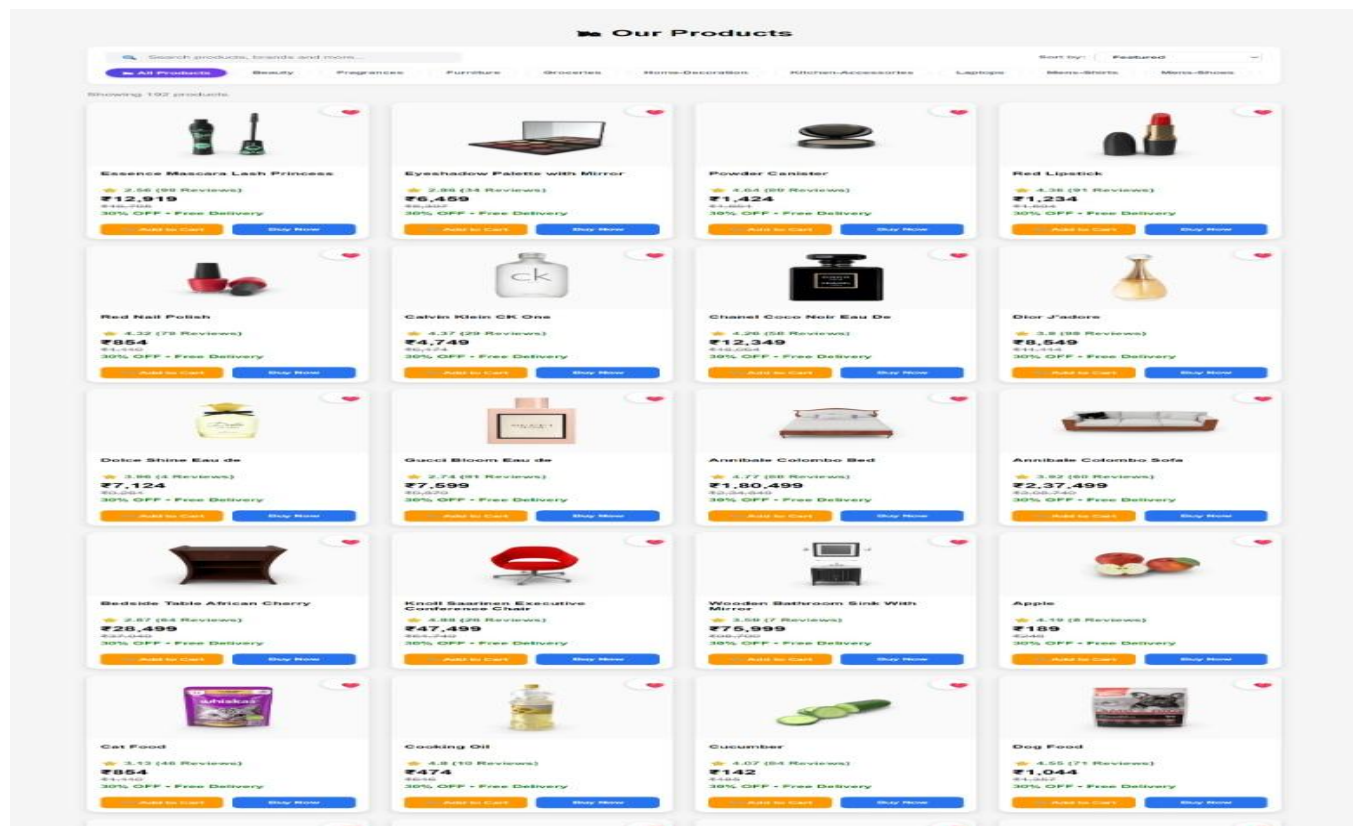
dharani@gmail.com

Register

Already have an account? [Login here](#)

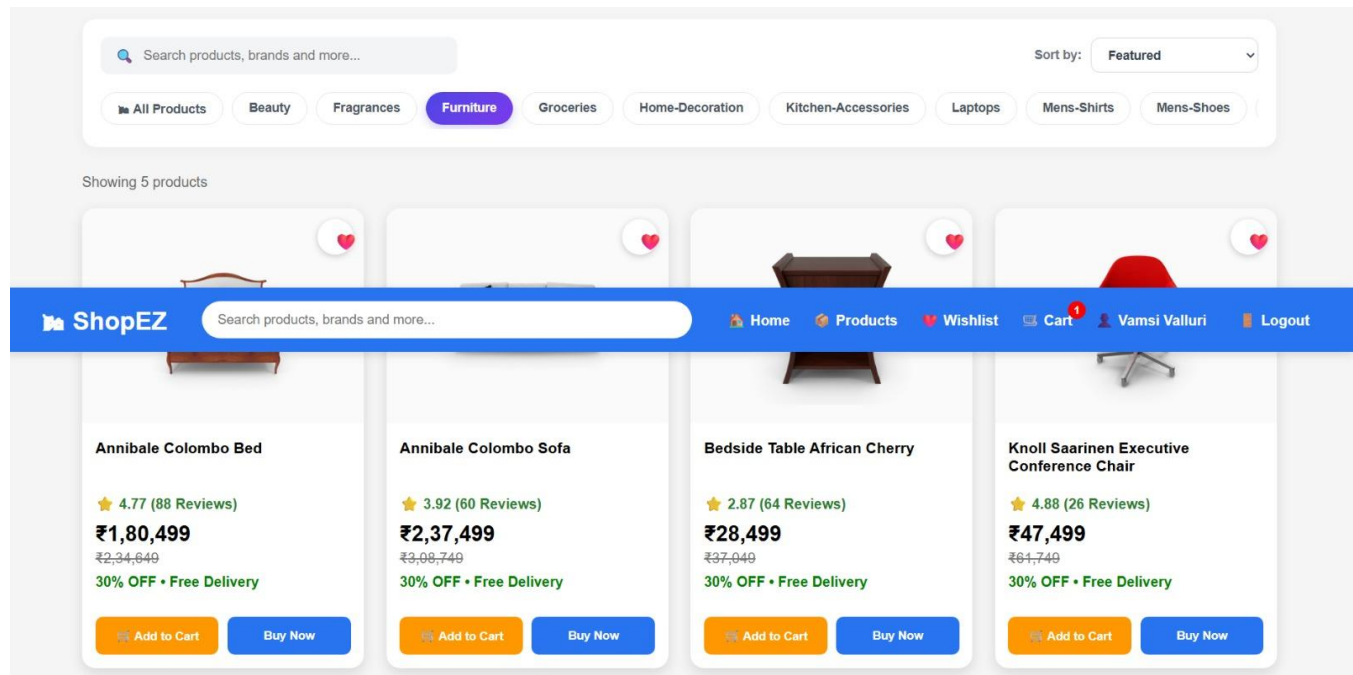
Product Listing Page

-Displays all available products with images, prices, discounts, and categories. Users can browse products and select items for detailed viewing.



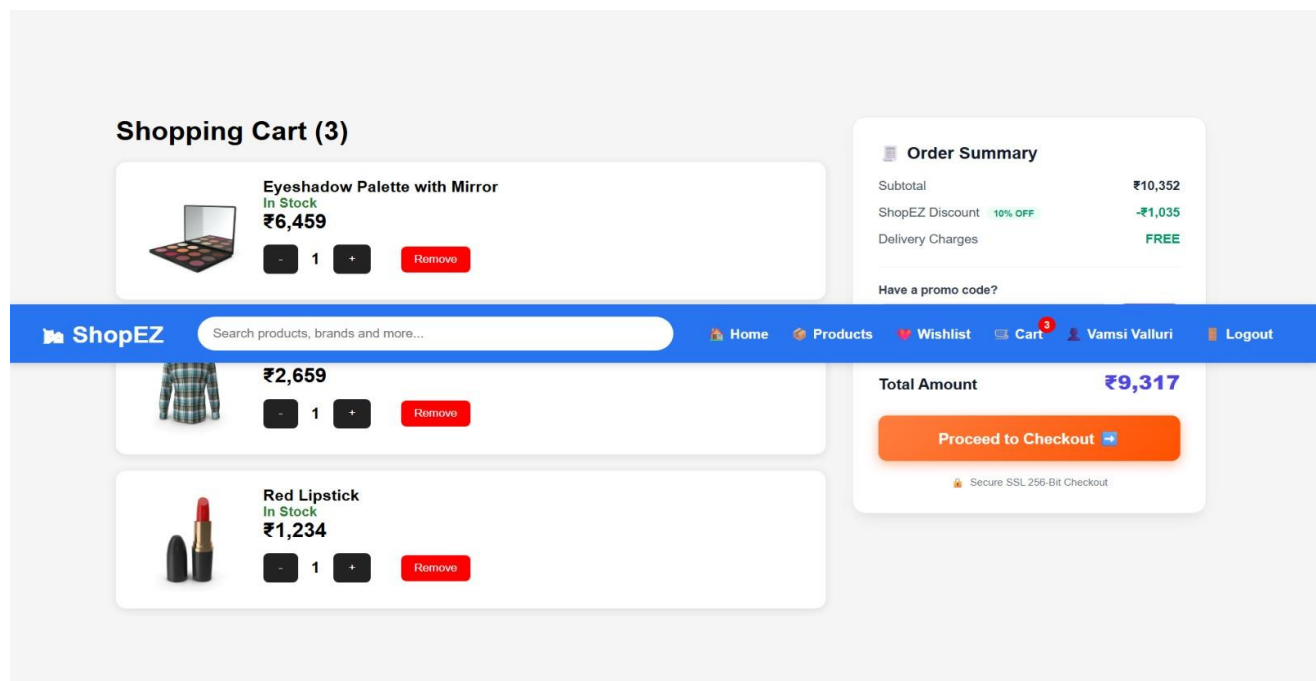
Product Details Page

-Provides complete information about a selected product, including images, description, available sizes, pricing, discounts, and the option to add the item to the cart.



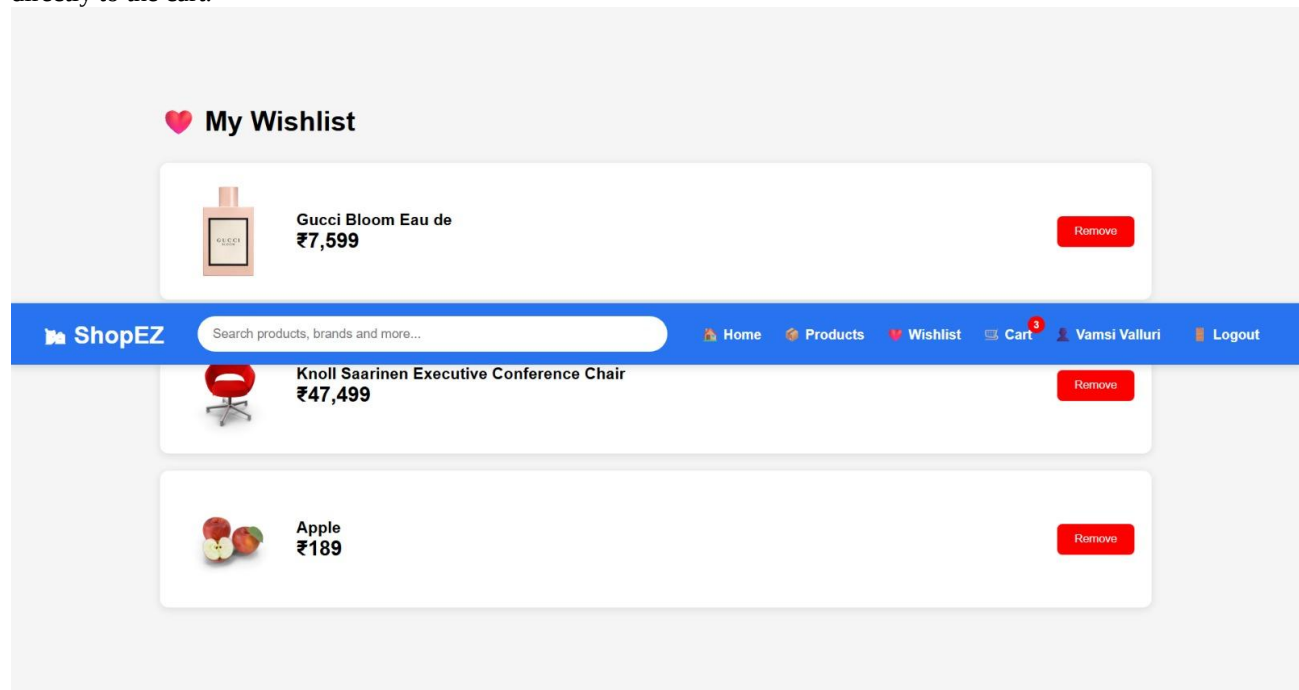
Shopping Cart Page

-Shows all products added by the user, including quantity, selected size, subtotal, and total amount before proceeding to checkout.



Wishlist Page

-The Wishlist Page allows users to save their favorite products for future purchases. Users can add or remove items from the wishlist, view product details, and quickly move selected items to the shopping cart whenever they are ready to buy. This feature enhances the shopping experience by helping users organize products they are interested in without adding them directly to the cart.



9. Summary

SHOPEZ demonstrates a complete MERN-stack e-commerce workflow — from user registration and product browsing to cart management and order placement — built on a clean MVC backend architecture that separates data (Model), business logic (Controller), and API routing (View). This documentation should serve as the primary reference for architecture decisions, database structure, and the development roadmap throughout the build process